

Portfolio Management on the JSE and NYSE: Integrating Deep Reinforcement Learning Trading Agents with Spatio-Temporal Graph Neural Network Encoders

Mehdi Jaffri

JFFSYE002@myuct.ac.za

University Of Cape Town

Cape Town, Western Cape, South Africa

Abstract

Markets are noisy which makes it difficult and complex for models to learn dynamics effectively. We aim to mitigate its effect by splitting the forecasting and portfolio management into separate components by utilizing an encoder-agent model structure. This paper tackles portfolio optimization management using DRL agents and STGNN encoders. We use DDQN, A2C and PPO trading agents with DNN, GWN and AGCRN encoders using walk-forward testing on the JSE and NYSE. We found that continuous agents were better suited to financial value optimization, and discovered that STGNNs preserved structural format and utilized temporal windows to produce better Calmar and Sortino scores. Our best models functioned as portfolio managers by enforcing stocks to fit a weighted allocation equivalent to an Index using a buy-and-hold strategy.

CCS Concepts

• **Computing methodologies** → **Reinforcement learning**; *Neural networks*; • **Information systems** → Spatial-temporal systems.

Keywords

proximal policy optimization, advantage actor critic, double deep q-network, Graph WaveNet, adaptive graph convolutional recurrent neural network

1 Introduction

The stock market is volatile and dominated by noise [2, 8], posing a challenge for price forecasting. While standard neural networks lack the functional depth to learn complex patterns from variable data [7], deep neural networks (DNNs) mitigate this constraint [2]. However, DNNs may not be sufficient as stocks do not fluctuate individually but instead evolve over time, where correlated stocks influence one another, potentially making forecasting difficult. To effectively learn these dynamics, models must also account for periodic and topological relationships that are inherent to stocks. Spatio-Temporal Graph Neural Networks (STGNNs) [16] address this by tracking temporal shifts and spatial dependencies, allowing models to theoretically learn complex market dynamics.

The model must develop a trading rule to maximize cumulative returns. The STGNN forecasts the prices of stocks, but it cannot execute trade decisions. Using deep reinforcement learning agents to forecast and learn effective trading rules at the same time increases complexity and training times, so alternatively the STGNN's encoded output serves as a rich set of features that is used by deep reinforcement learning agents to learn optimal trading policies.

This paper splits the trading framework into two components: feature extractor (the "encoder") and an execution policy (the "agent"). This paper then defines the combination of an encoder and agent as a "model" as shown in Figure 4 in the appendix, where this model architecture attempts to achieve results and compares them to market baselines.

To evaluate the generalisability of the findings, models are benchmarked using Johannesburg Stock Exchange (JSE) and New York Stock Exchange (NYSE) datasets. The NYSE is a mature dataset with extensive benchmarking in literature [6, 14, 17], while the JSE is an emerging market with less mainstream use in literature [11].

1.1 Problem Statement

The DNN is a simple feed-forward neural network that requires a flattened 1D array. Its input features match the format $[n \times F]$ where n is the number of stocks, and F is the features per stock. Alternatively, STGNNs are spatio-temporal layers over a graph, defined as a set of nodes and edges $G = (V, E)$, where V represents the set of stocks and E describes the relationship between each pair of stocks. The input features are formatted as $[T_w, n, F]$, where T_w is the number of days in the window history.

The DNN is only provided the current day's Features F , while the STGNN is given all features for all days in T_w . The DNN encoder cannot be fed T_w days of data, even with flattening, as the number of weights would explode which makes it computationally too expensive to use or train [3].

The STGNN receives data with its topological format intact, while DNN requires reformatting it by flattening. This dimension reduction can cause important relationships to be missed, hence the need for STGNN for potentially richer feature extraction [16].

The Markov Decision Process is defined as (S, A, P, r, γ) . The STGNN outputs state data S using all the information we have available. The DNN needs the data to be flattened which removes the spatial aspect. The DNN can then only process the flattened data to output state S_{flat} . S_{flat} cannot be better than S on average, as S_{flat} was created with degraded spatial and temporal data and therefore S_{flat} has degraded information compared to S . With degraded state data, the agent's decision capabilities suffer and it converges on a suboptimal policy.

Finally we use raw data from the New York Stock Exchange (NYSE) and the Johannesburg Stock Exchange (JSE). The JSE represents an emerging economy with lower trading volumes and higher variance. Conversely, the NYSE provides a mature and highly liquid market with larger trade volumes. Therefore, due to market differences in volatility and liquidity, it cannot be assumed that

a model that performs well on one market will perform well on another. Only by benchmarking performance on both datasets can we determine that the model is robust at learning effective policies in different market dynamics.

Research Questions:

- How do DRL-based share-trading agents (DDQN, A2C, and PPO) perform in terms of key metrics when compared against a simple rule-based trader, when using DNN and STGNN-based state encoders (Graph WaveNet and AGCRN), using datasets from the JSE and NYSE?
- Is there a significant performance difference in continuous DRL algorithms such as PPO and A2C vs discrete DRL algorithms such as DDQN based on key metrics for stock trading over the JSE and NYSE?

1.2 Research Contribution

- Designed a modular Environment, Encoder and Agent framework that can be extended or reproduced providing a structural framework for executing agents using different encoders on the datasets.
- Acts as one of few early tests of STGNNs with DRL agents on the JSE providing literature on the local South African market.
- Comparative analysis of Encoders and Agents provide insight into their efficacy and the trading strategies that emerge.

2 Background

Deep reinforcement learning (DRL) algorithms were introduced for portfolio management by early pioneering papers [4] and demonstrated effective policy learning while presenting training limitations in convergence due to noise. Simultaneously learning forecasting and portfolio optimization becomes too complex, so split architectures were used to simplify each component and reduced individual complexity [13]. While DRL agents were effective at policy generation to maximise reward [4, 6], they fail to sufficiently extract features requiring the task be handled separately.

Deep Neural Networks (DNNs) use hidden layers for learning deep patterns but scaled poorly for noisy temporal data [2, 7]. DNNs imposed structural degradation of data into linear arrays [2] to function. Spatio-Temporal Graph Neural Networks (STGNNs) made use of data with intact spatial topology revealing deeper patterns which led to better models. A foundational STGNN, Graph Wavenet (GWN) was initially used on a traffic dataset [16], but was also applied to financial markets, due to similarities in format and noise. Papers found GWN effective at forecasting in financial datasets [11]. Early GWN depended on static graph parameters but newer models, such as Adaptive Graph Convolutional Recurrent Network (AGCRN), do not need to follow that strict format [1] and were also used due to their superiority in extracting inter-stock relationships.

While benchmarking these models on the NYSE is common [6, 14, 17], due to its nature as a mature market, we also make use of the JSE [11] which has a relative lack of literature as an emerging market.

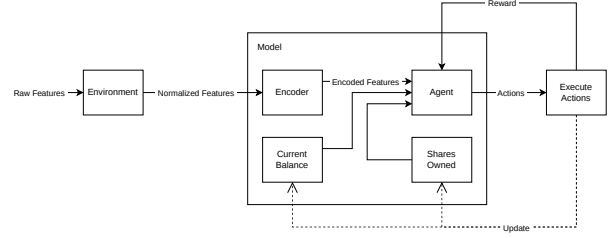


Figure 1: DRL agent pipeline training pipeline

3 Experimental Design

3.1 Problem formulation

We previously defined the Markov Decision Process as (S, A, P, r, γ) . We need to explicitly define each value for our agent to function in the pipeline as shown in Figure 1. We address each component individually:

3.1.1 State Format (S). The encoder outputs the forecasted returns as vector E_t where t is the current timestep. For the agent to understand the entire environment framework, we must also provide the number of shares held for each stock and the balance.

We define N_t as the number of shares owned for each stock and b_t as the remaining balance at the start of timestep t . We limit each element of N_t to hold at most 1000 shares per stock. 1000 was chosen as a sufficiently large amount without causing the model to hold any significant part of the market. The encoder outputs values between $[-1, 1]$. The raw values of N_t and b_t are too large for the agent to use directly, so they must be scaled. We scale them as $\bar{N}_t = N_t / 1000$ for share count and scale the balance as $\bar{b}_t = b_t / b_0$ where b_0 is the initial balance at the start of the trading period. The scaled values lie around 1 to prevent gradients exploding and maintain convergence stability.

Therefore state S_t at timestep t is characterized as E_t from the encoder, representing current market forecast, vector $\bar{N}_t$ which holds the scaled number of owned shares and a scalar $\bar{b}_t$ value that shows the scaled current balance, all concatenated together for a total state length of $2n + 1$.

3.1.2 Action Space (A). The trading agents are constricted to either discrete or continuous agents, which determines their action domain. At timestep t , the model forwards the input state S_t and outputs a vector A_t that contains n values. Each value determines the action to take for that given stock.

- **Discrete Action Space:** Value based agents such as the Double Deep Q-Network (DDQN) agent, output an action vector $A_t \in \{0, 1, 2\}^n$. Given that there are n stocks, and $0 \leq i < n$, $i \in \mathbb{Z}$.

$$A_{t,i} = \begin{cases} 0, & \text{Hold position} \\ 1, & \text{Buy position} \\ 2, & \text{Sell position} \end{cases} \quad (1)$$

The discrete agent sells shares of all valid stocks where $A_{t,i} = 2$ is true. We then buy shares for each valid stock where $A_{t,i} = 1$ is true. Purchasing power is equally split across all stocks. We limit any buy/sell transaction to a

maximum of 100 shares per day for any given stock. This is to enforce that the agent cannot buy or sell too many shares in a panic.

- **Continuous Action Space:** Policy-gradient architectures like Advantage Actor-Critic (A2C) and Proximal Policy Optimization (PPO), output an action vector $\mathbf{A}_t \in [-1, 1]^n$. For each stock, a value > 0 is associated with buying while < 0 is associated to selling and $= 0$ is equivalent to holding. We scale 1 to 100 shares at most. Given that there are n stocks, and $0 \leq i < n$, $i \in \mathbb{Z}$. The continuous agent first sells all valid stocks where the action is $\mathbf{A}_{t,i} < 0$. It sells up to $\mathbf{A}_{t,i} \times 100$ shares given that we have enough shares to sell. The agent then buys all valid stocks where the action is $\mathbf{A}_{t,i} > 0$. It buys up to $\mathbf{A}_{t,i} \times 100$ shares given that we have space for the shares and enough money. The money is split based on the $\mathbf{A}_{t,i} \odot \mathbf{P}_{Open,t,i}$ ratio where $\mathbf{P}_{Open,t,i}$ is the opening price of a given stock's share at timestep t .

The appendix contains a worked example of the calculations for each action space, for those who are interested.

3.1.3 Reward Value (r). The continuous and discrete trading agents calculate a reward r_t for that timestep. The value is calculated as $r_t = v_t/v_0$ where v_t is the portfolio value after executing the action at timestep t and v_0 is the initial starting portfolio value (which is equivalent to b_0 , which is the initial balance.) At any given timestep t , portfolio value is defined as $v_t = \mathbf{P}_{Close,t} \odot \mathbf{N}_{t+1} + b_{t+1}$, where $\mathbf{P}_{Close,t}$ is the closing price for each stock at the end of trading day at timestep t . We use cumulative returns from v_0 as it keeps the reward value stable and around 1, rather than daily change where $r_t = v_t/v_{t-1}$ could cause the gradients to explode in edge cases.

3.1.4 Transition Probability (P). P is not modelled and unknown. The DRL agents used are resilient and do not require explicit probabilities.

3.1.5 Discount Factor (γ). We use a discount factor of 0.99 for all agents.

3.2 JSE and NYSE Preprocessing

The first step in our pipeline in Figure 1, is to setup our environment using the raw feature data. The environment uses daily market data from 1st January 2009 to 31st December 2025, totalling 17 years for the JSE and NYSE datasets. We first filter for the stocks in the Top40 Index for the JSE and the Dow Jones Index for the NYSE. This introduces survivorship bias, but reduces the computational complexity required to process many stocks. The datasets contain daily stock data with the date, stock token, open price, close price, high price, low price, volume and total return. We also extract the daily value of the Index for each dataset, which we use for comparison in Section 4. The raw values are stored as cents, so a stock that costs R8.42 is stored as 842. We make use of this raw cent value and all balances are stored as cents to simplify things. We remove any days with missing data and then split the data into Train, Validation and Test sets as shown in Table 1.

We then filter each stock that has fewer than 3 years in the train set. We convert the raw values to relative changes. The data is formatted to output 7 features for each stock per day. These features are:

Set	Year range	Number of years
Train	2009-2018	10
Validation	2019-2021	3
Test	2022-2025	4

Table 1: Chronological Data Splits for the JSE and NYSE datasets

- Relative open price change - the percentage change in open price compared to the previous day's close price.
- Return - the total return for the previous day.
- Relative high price change - the percentage change in the previous day's high price compared to the previous day's open price.
- Relative low price change - the percentage change in the previous day's low price compared to the previous day's open price.
- Relative exponential weighted moving average (EWMA) volume - the percentage change in the previous day's trade volume compared to a EWMA volume.
- Garman-Klass volatility - the stock's price volatility during the day calculated using the previous day's high and low prices.
- EWMA Price volatility - the stock's price volatility over time using EWMA volatility.

We make the features into relative changes to combat the magnitude growth that exists in finance. This way a growth of 5% in the first year, is equivalent to a growth of 5% in a later year. Making the changes relative does remove some degree of information, so we also provide the Garman-Klass volatility, which provides the price volatility of the stock during the day, and the EWMA price volatility which is the stock's price change over time. The return is kept raw as it is already formatted to a percentage change (a return of 0.1 means 10% increase in share value).

Once the features have been made relative, we process them and normalize them using the train set's features. The Garman-Klass volatility and the EWMA are min-max normalized, while the relative open, high, low, volume and return are z-score normalized.

We then format each dataset into $[t, N, F]$, where t is the timestep, N is the stocks in the dataset, and F is the features for each stock. At the same time, we generate a mask that determines whether a stock is available to trade or not in the format of $[t, N]$ where each value is either 0.0 or 1.0 which determines whether the stock is unavailable or available respectively.

The environment also formats the features into the correct format that the encoder expects. For STGNN encoders it outputs batches in the format of $[b, t, N, F]$, where b is the current batch, t is the timestep, N is the stock and F is the features for the given stock on that timestep. For DNN encoders it first flattens the nodes and features and then outputs batches in the format of $[b, t, N \times F]$. The STGNN encoder data is formatted with a window length of 20 days. Both encoders use data formatted for 1 day horizon and use a stride of 1 day. A batch size of 32 is used alongside a transaction fee of 0.25%.

3.3 Feature Encoders

After the environment has processed the data, the next step in the pipeline shown in Figure 1 requires the encoder to use the data to output encoded returns.

We make use of three encoders, namely DNN, GWN and AGCRN. The GWN and AGCRN are both STGNN encoders and rely on different input format compared to the DNN. The environment is setup to handle this format difference by specifying which format is required.

When training, we iterate through the loader’s batches for each seed. The train set loader is shuffled for encoder training. We train on each batch and update accordingly. For each encoder, across all 5 seeds, we save the model with the lowest loss.

3.3.1 DNN Encoder. The DNN encoder is a feed-forward neural network, implemented using pytorch modules. The input layer expects $n \times F$ values, where n is the number of stocks and F is the number of features per stock. The DNN has 3 hidden layers with sizes [256, 128, 64], and an output layer with size n , where each value is the return for the given stock at that given timestep and a dropout before every layer of 0.1. We use learning rate of 0.0003 and weight decay of $1e - 5$. We train the DNN using MSE loss on the train set and validate every 5 epochs. We use 5 different seeds, which are [42, 101, 777, 999, 2026] and run each seeded model for 200 epochs.

3.3.2 STGNN Encoders. The STGNN encoders use implementations from TSL for GWN and AGCRN. The input layer expects data in the format $[t, n, F]$, where t is the timestep, n is the number of stocks and F is the number of features per stock. The STGNNs have input size $F + 1$ (previous day’s return is repeated due to STGNN input format requirements) and output size 1, which is the current day’s return, for each node. The AGCRN learns adjacency rules by default, and the GWN is configured to learn them as well using an $n \times n$ matrix of ones as the initial edge weights. They both use embedded size 12 and hidden size 32 to learn the adjacency weights. We use learning rate of 0.0003 and weight decay of $1e - 5$. We train the STGNN using MSE loss on the train set and validate every 5 epochs. We use the same 5 seeds as the DNN, which are [42, 101, 777, 999, 2026] and run each seeded model for 200 epochs for GWN and 400 epochs for AGCRN. AGCRN is run for 400 epochs as it does not converge in 200.

3.4 DRL trading agents

The next step of the pipeline, in Figure 1, involves the encoder giving the encoded returns to the trading agent. Before the agent starts trading, we pre-compute the encoded data using the frozen weights from the stable encoder’s model. The agent owns its own balance and the shares and must manage its memory.

At the start of training, we use the encoder with frozen weights to encode all features and save them. At the start of the epoch, the model initializes its own balance and number of shares owned. The model then iterates through each timestep and provides the current state, concatenating the encoded data, the shares owned and the current balance, which is accepted by the agent, who outputs a vector of actions. These actions dictate the action to be taken for each stock, and the agent fulfils them to the best of its ability and

updates balance and shares owned accordingly. Using cumulative return as the reward for the agents, we update the weights and move to the next timestep.

This is repeated for each Agent, using each encoder across all 5 seeds which are [42, 101, 777, 999, 2026]. The agents are trained for 750 epochs and are validated at every 10th epoch.

The DRL agents use $1e - 5$ weight decay and 0.99 for the reward discount factor. Each agent uses logged Huber loss for back propagation and a dropout rate of 0.2 at every layer.

3.4.1 Rule-based Trading agent. The baseline rule-based trading agent checks the encoded output and purchases when the prediction exceeds a certain threshold. The threshold used is 0.35. Therefore, we buy when the prediction exceeds 0.35, we sell if the prediction falls below -0.35 , and we hold if the magnitude lies in range $-0.35 < a_{t,i} < 0.35$, where a is the action, t is the timestep and i is the stock index.

3.4.2 Double Deep Q-Network Trading agent. Double Deep Q-Network (DDQN) [15] updates its weights by reducing the mean squared Bellman error: $\mathcal{L}(\theta) = \mathbb{E}_t[(r_t + \gamma \max_{A'} Q(S_{t+1}, A'; \bar{\theta}) - Q(S_t, A_t; \theta))^2]$, where $\bar{\theta}$ represents the parameters of the target network. DDQN uses an experience buffer that only starts updating the weights once the buffer has 1000 days of experience and stores a maximum of 50000 days of experience, replacing the oldest when overflowing. The DDQN is also limited to only update weights every 5 timesteps and samples using batch size 32 and learning rate of 0.0005. DDQN starts with exploration epsilon at 0.9 and exponentially decays by 0.01 every epoch. DDQN’s target DQN is updated at the end of each epoch and both target and policy DQN are feed-forward neural networks using input layer size $2n + 1$, where n is the number of stocks. DDQN’s policy and target have output layer size $3n$ which is 3 Q-value scores per stock. We process the scores by taking the highest to get one action per stock. The policy and target have 3 hidden layers with sizes [128, 128, 128].

3.4.3 Advantage Actor-Critic Trading agent. Advantage Actor-Critic (A2C) [9] uses a replay buffer that updates weights once the buffer has filled with 64 days of experience and removes them all after updating weights by replaying memories in the buffer using learning rate 0.0005. A2C manages its own exploration using the Actor-Critic design updating its weights by minimising its loss: $\nabla_{\theta} \mathcal{L}(\theta) = \mathbb{E}_t[\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \bar{a}_t]$, where $\bar{a}_t$ is the scalar advantage calculated by the critic network. Both actor and critic are feed-forward neural networks using input layer size $2n + 1$, where n is the number of stocks. The actor output layer size is $2n$ which is the mean and standard deviation of the current action distribution for each stock, and 3 hidden layers with sizes [128, 128, 128]. The critic has an output layer size of 1 which describes the overall value of the next state, and 3 hidden layers with sizes [128, 64, 32].

3.4.4 Proximal Policy Optimization Trading agent. Proximal Policy Optimization (PPO) [12] uses a memory buffer that updates weights once the buffer has filled with 256 days of experience and removes them all after updating weights with a batch size of 64. PPO uses 0.00011 for the DNN and GWN encoders, and 0.00005 for the AGCRN encoder for the learning rate. PPO fluctuates rapidly and collapses without finding an effective policy,

when learning rate is too high, therefore these values were used as they were found to produce stable results. PPO manages its own exploration using the Actor-Critic design alongside a clipping objective function to stabilize weight updates: $\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(p_t(\theta)\bar{a}_t, \text{clip}(p_t(\theta), 1-\epsilon, 1+\epsilon)\bar{a}_t)]$, where $p_t(\theta)$ is the scalar policy probability ratio and ϵ is the clipping value. PPO updates its actor and critic's weights when replaying the memories in the replay buffer in the rollout epochs. PPO replays the buffer for 2 rollout epochs when using the AGCRN encoder and 4 rollout epochs when using the DNN or GWN encoder and using an epsilon clip of 0.2 and entropy coefficient of 0.01. Both actor and critic are feed-forward neural networks using input layer size $2n + 1$, where n is the number of stocks. The actor output layer size is $2n$ which is the mean and standard deviation of the current action distribution for each stock, and 3 hidden layers with sizes [128, 128, 64]. The critic has an output layer size of 1 which describes the overall value of the next state, and 3 hidden layers with sizes [128, 64, 32].

3.5 Model Selection Criteria and Metrics

During encoder training, we save the model with the lowest MSE loss. We save and use only one encoder model across all of our seeds which all DRL agents use. This means we have a total of six encoder models that we use across datasets and feature encoders.

For DRL training, we save and use the model with the highest *Calmar Ratio* $\times$ *Sortino Ratio* for each seed. Calmar ratio is defined as $\text{Calmar Ratio} = \frac{R_p}{\text{Maximum Drawdown}}$, where R_p is the average annualised return from $R_p = \text{ROI}^{\frac{1}{4}} - 1$ since there are 4 years in the test set and $\text{Maximum Drawdown} = \max_{t \leq T} \left(\frac{V_t - V_t^*}{V_t} \right)$ is the maximum drop observed in our portfolio value. Sortino ratio is defined as $\text{Sortino Ratio} = \frac{R_p - R_f}{\sigma_d}$ where R_p is the average annualised return, R_f is the target/risk-free return which we have set to the Top40 Index's return for the JSE and the Dow Jones' return for the NYSE, and σ_d which is the downside deviation. We define downside deviation as $\sigma_d = \sqrt{\text{mean}((R_t - R_{f,t})^2) \times \sqrt{\text{Number of days in a year}}}$, where R_t is the model's return at timestep t and $R_{f,t}$ is the target return required to beat the Index. We use the Index as a high-performance target so expect small or even negative values as positive values would indicate that we beat the Index and were more profitable with the same amount of downside risk. We make use of these metrics since they inform us on how the model performs and how robust it is at obtaining the ROI it gets, by checking the risks it takes and how profitable those risks are. Sortino and Calmar are specifically chosen due to them isolating downside risk unlike an alternative such as Sharpe ratio which penalizes positive volatility as well. *Calmar Ratio* $\times$ *Sortino Ratio* also has no mathematical value, but it serves as a basic way to filter for high scores in both metrics which should be more robust compared to a model that only has a high score in one metric. Doing this for all RL agents, seeds and encoders across both datasets results in ninety agents saved for the project. When data is averaged across seeds, we have eighteen configurations to compare across two separate datasets.

All agents and encoders have been tuned for stability, not performance. We attempt to compare their results using the as-is implementations rather than explicitly tuning each to their best capabilities.

3.6 Walk-Forward Testing

Once our models have been saved, we evaluate them using the test set. We do this by evaluating each year separately, resetting the balance and shares owned each year. We then train on one year and then use the updated model to evaluate the following years. For example, if there are three years: year 1, year 2 and year 3, we first evaluate our model on all three years. Then we train on year 1. We then evaluate on only year 2 and 3. Then train on year 2 and evaluate on year 3. By doing this we achieve walk-forward testing [5, 10] where the model can fine tune to the current market dynamics using more recent data without having to relearn everything for trading.

The models train on the test year for 12 epochs. They use the same training structure and rules as the model training described in Section 3.4 but differ in some regards which are described below. All models use a initial balance of 1000000 and DRL agents have been tuned so that walk-forward is stable to ensure a fair comparison.

We load the model weights for the model for each seed, reset the seed and then start training the model, saving the weights after every training step. We evaluate the model at the start of each year and then train it on that year before evaluating the next year. This way we end up with four models for one configuration of seed, encoder and RL agent. The PPO and A2C models both have a critic and actor component which are both saved, but we count that as one model for the sake of simplicity. During training, the balance and owned shares are reset every epoch, but during the final evaluation, we use a running balance that is passed through each year alongside a running portfolio of stocks that evaluates each year using the model trained right up to before that year. This outputs a final ROI, Sortino ratio and Calmar ratio for each agent, encoder and seed configuration. We also track the number of shares traded in each configuration for any given stock in a timestep.

3.6.1 DDQN Walk-Forward Testing. For DDQN we have dropped learning rate from 0.0005 to 0.000005 and it now updates every step rather than every 5th step. We increased weight decay from $1e-5$ to $1e-4$ and reduced the min and max replay buffer size from [1000, 50000] to [500, 5000]. We also initialise exploration epsilon at 5% rather than 90% and set the decay from 0.99 to 0.999 per epoch and increase the minimum epsilon from 0.0001 to 0.01. We update the target every 50 steps rather than every epoch and the training buffer persists when training on the following years. This makes the DDQN learn slowly and fine-tune itself to the current market dynamics. The persistent buffer is used to keep the model from majorly overfitting to the latest year's market by introducing memories of previous years.

3.6.2 A2C Walk-Forward Testing. For A2C we kept learning rate, the trajectory buffer and weight decay the same. The model's initial training resulted in the critic being highly confident which inhibits learning new techniques in favour of actions that are deemed as optimal. This was addressed by introducing entropy to the A2C agent's policy, which we did with an entropy coefficient of 0.05.

3.6.3 PPO Walk-Forward Testing. For PPO we drop the learning rate used for DNN and GWN encoders from 0.00011 to 0.00005. We also drop the clipping epsilon from 0.20 to 0.15 and raise the entropy coefficient from 0.01 to 0.05.

4 Results

Table 2 and Table 3 present our models’ walk-forward performance, on the test set, for the JSE and NYSE respectively. Each encoder-agent model configuration, presents the ROI, Sortino ratio and Calmar ratio. ROI contains the percentage change averaged across the seeds alongside one standard deviation. Sortino ratio is calculated during evaluation and the minimum acceptable return used is the Index’s return for each respective dataset. Sortino and Calmar are reported in the format *median* [*min*, *max*] across the seeds. The best value of each metric is bold, while the second best is underlined.

On the JSE, the rule-based trading agent did not make any trades and therefore had no net change for each encoder. On the NYSE, the rule-based trading agent did not make any trades for the STGNN encoders, but did for the DNN encoder. It ended up being the only model to lose money with a ROI of -12.50% . The Index’s results have also been added as we use it as an alternative comparison target. The peak ROI observed on the JSE was $98.2\% \pm 53.5\%$ using the AGCRN-A2C model and the highest ROI observed on the NYSE was the GWN-PPO model with $49.8\% \pm 13.0\%$.

5 Analysis and Discussion

5.1 Comparative Analysis of Trading agents

5.1.1 Discrete vs Continuous Agents. On the JSE dataset, DRL models achieved ROI ranging from $24.50\% \pm 15.20\%$ up to $98.20\% \pm 53.50\%$. Given that the Top40 Index achieved an ROI of 61.04% over the test set, only three out of nine models beat the Index. These are DNN-A2C, AGCRN-A2C and AGCRN-PPO. All three used continuous agents, and the models that used discrete agents achieved up to $50.10\% \pm 22.70\%$, which only had some seeds beat the Index, but on average performed worse than the Index. On the NYSE dataset, DRL models achieved ROI ranging from $12.90\% \pm 10.40\%$ up to $49.80\% \pm 13.00\%$. Given that the Dow Jones Index achieved an ROI of 31.37% over the test set, six out of nine models beat the Index. These six were the models that use A2C and PPO DRL agents. Across both datasets, DDQN performs worse than continuous agents such as A2C and PPO across all encoders. The discrete agent gets lower ROI, lower median Sortino for two out of three encoders on the JSE and all encoders on the NYSE, and lower median Calmar for all encoders and datasets.

This performance drop occurs due to the nature of discrete actions inherent to the DDQN agent. When it chooses to buy, its rigid action space forces it to allocate funds uniformly to all stocks, preventing the agent from making more nuanced transactions. This lack of flexibility causes it to be more variable and inefficient at portfolio management. Alongside this, DDQN is a value-based DRL which makes it more susceptible to noise, through estimation bias, and overfitting to current market regimes unlike A2C and PPO which stabilize weight updates.

5.1.2 Policy Convergence Differences: A2C vs. PPO. For both datasets, A2C agents outperform PPO agents for DNN and AGCRN encoders in terms of ROI. On the JSE, PPO agents have better median Calmar and Sortino scores compared to A2C for the GWN and AGCRN encoders. On the NYSE, PPO has better median Sortino and Calmar scores for GWN, while A2C has better scores for DNN. For the AGCRN encoder, on the NYSE, A2C has a higher Sortino ratio but

PPO has a higher Calmar ratio. On the NYSE, for both A2C and PPO, Calmar lies in range $[0.422, 0.532]$, which is a small range, indicating that the models have similar performance in that metric for both agents across all encoders. All A2C and PPO models also outperformed the Index in Calmar scores. Sortino on the other hand is more variable, but higher for PPO ranging $[0.095, 0.242]$, and less variable but lower score for A2C ranging $[0.026, 0.107]$. On the JSE, A2C’s Calmar scores are worse on average ranging $[0.384, 0.569]$ where PPO is as variable but higher ranging $[0.519, 0.701]$. Sortino on the other hand is more similar, but PPO is slightly higher for AGCRN. Median scores for Sortino also indicate that the models outperformed the Index, but the difference varies per model. Given all these details, even though an A2C agent was the best model in terms of ROI on the JSE, in terms of finance metrics like Sortino and Calmar, PPO is better. Similarly, on the NYSE, A2C gets lower finance scores, with higher ROI compared to PPO on average for two out of three encoders, but PPO ends up as the agent with the highest peak ROI with the third encoder. Interestingly, The model with the best ROI, Calmar and Sortino score on the NYSE was the GWN-PPO model, while the second highest for each metric was the DNN-A2C model. On the JSE, The AGCRN-A2C had the highest peak ROI, the DNN-DDQN had the highest Sortino ratio while the AGCRN-PPO model had the best Calmar score, the only model with a Calmar score higher than the index, and also had the second highest ROI and Sortino score. GWN-PPO had the second highest Calmar score on the JSE, indicating that while A2C might have the highest peak ROI, PPO had more stable and effective models in terms of Calmar and Sortino while still achieving competitive ROI.

This difference between A2C and PPO occurs due to their method of updating weights. A2C makes bigger updates to its weights without clipping, which makes it take more risks and achieve the higher ROI in general, but lose out in the Calmar and Sortino scores, while PPO which updates its weights slowly using its epsilon clipping, isn’t always able to adjust as quickly and take those risks for ROI, but still manages to achieve better finance metrics. This effect can also be further understood, by viewing the JSE as a more variable market compared to NYSE. The JSE has bolder and bigger shifts which the A2C is better at catching, while the NYSE has smaller shifts that the PPO excels at with its clipping, resulting in the A2C succeeding on the JSE and the PPO succeeding on the NYSE in terms of peak ROI, but PPO was observed as the better DRL agent for Calmar and Sortino on both datasets.

5.2 Feature Encoder Evaluation

5.2.1 Flattened feature vs. Windowed-Node format. DNN’s performance was better than the GWN on both datasets, for the DDQN and A2C DRL trading agents. DNN only beats AGCRN for DDQN on the JSE, and only with A2C on the NYSE. The only rule-based trading model that actively interacts with the trading simulation, is the DNN encoder configuration on the NYSE, and that is also the only model that loses money and ends up with less money than the initial amount with a final ROI of -12.50% . None of the best or worst ROI models use DNN and instead the worst models use GWN while the best use GWN and AGCRN. Compared to GWN and AGCRN, the DNN generally lies in the middle.

State Encoder	Metric	Top40	Rule-Based	DDQN	A2C	PPO
DNN Encoder	ROI	61.04%	0.0%	50.1% $\pm$ 22.7%	78.1% $\pm$ 14.6%	55.8% $\pm$ 9.6%
	Sortino	0.000	na	0.338 , [-0.308, 0.465]	0.025, [-0.04, 0.255]	-0.158, [-0.262, 0.023]
	Calmar	0.646	na	-0.05, [-0.259, 0.102]	0.569, [0.230, 0.708]	0.519, [0.263, 0.742]
GWN Encoder	ROI	61.04%	0.0%	24.5% $\pm$ 15.2%	50.8% $\pm$ 30.3%	56.3% $\pm$ 20.9%
	Sortino	0.000	na	-0.359, [-0.413, -0.196]	-0.142, [-0.324, 0.271]	-0.060, [-0.335, 0.144]
	Calmar	0.646	na	0.142, [0.030, 0.365]	0.335, [0.008, 0.856]	<u>0.605</u> , [0.177, 0.806]
AGCRN Encoder	ROI	61.04%	0.0%	32.0% $\pm$ 10.8%	98.2% $\pm$ 53.5%	78.9% $\pm$ 13.0%
	Sortino	0.000	na	-0.219, [-0.416, -0.167]	0.045, [-0.130, 0.508]	<u>0.117</u> , [-0.097, 0.215]
	Calmar	0.646	na	0.203, [0.053, 0.209]	0.384, [0.192, 0.783]	0.701 , [0.443, 1.075]

Table 2: JSE Test Set Walk-Forward Results. Best ROI Value is in bold, while the second highest is underlined.

State Encoder	Metric	Dow Jones	Rule-Based	DDQN	A2C	PPO
DNN Encoder	ROI	31.37%	-12.5%	22.2% $\pm$ 8.0%	<u>43.2% $\pm$ 8.9%</u>	39.6% $\pm$ 9.7%
	Sortino	0.000	na	-0.134, [-0.249, -0.026]	<u>0.107</u> , [-0.057, 0.256]	0.095, [-0.086, 0.217]
	Calmar	0.352	na	0.220, [0.113, 0.358]	<u>0.506</u> , [0.366, 0.604]	0.452, [0.297, 0.529]
GWN Encoder	ROI	31.37%	0.0%	12.9% $\pm$ 10.4%	39.3% $\pm$ 9.1%	49.8% $\pm$ 13.0%
	Sortino	0.000	na	-0.368, [-0.377, -0.064]	0.026, [-0.066, 0.209]	0.242 , [-0.039, 0.314]
	Calmar	0.352	na	0.053, [0.028, 0.304]	0.422, [0.340, 0.526]	0.532 , [0.347, 0.620]
AGCRN Encoder	ROI	31.37%	0.0%	31.0% $\pm$ 11.6%	37.0% $\pm$ 9.0%	36.8% $\pm$ 4.9%
	Sortino	0.000	na	-0.088, [-0.191, 0.172]	0.046, [-0.129, 0.153]	0.018, [-0.059, 0.123]
	Calmar	0.352	na	0.263, [0.135, 0.380]	0.436, [0.254, 0.465]	0.458, [0.348, 0.487]

Table 3: NYSE Test Set Walk-Forward Results. Best ROI Value is in bold, while the second highest is underlined.

This shows that the DNN does have sufficient information, with flattened features in the format $[n \times F]$, to perform well on the market, since it was able to achieve results above the Index on both datasets, but also shows that encoders that preserve structure, with a lookback window and intact node structure in the format $[T, n, F]$, can perform better. Two out of three DNN using DRL agents have the lowest Calmar (third is A2C which has the best) on the JSE, and its Sortino value is highest for DDQN, second for A2C and worst for PPO on the JSE. DNN occupies different rankings for each agent, making it quite variable on the NYSE.

DNN generally has worse scores for Calmar and Sortino, and only the best DNN models beat the worst or average STGNN models. On average STGNN's have better ROI performance and higher observed Sortino and Calmar scores. This discovery serves as evidence that the STGNN's window and node preserving format is able to lead to a higher quality result but depends on the STGNN implementation used.

5.2.2 STGNN Learned Relationships. Figure 2 shows the adjacency matrices of the STGNN encoders on each dataset. A and C show the AGCRN's learned relationships, while B and D use the GWN to learn instead. A and B show the relationships learnt on the JSE, whilst C and D are the equivalent on the NYSE. At a cursory glance AGCRN has produced a matrix with distinct values, while

the GWN's matrix has uniform values, for both datasets. Each cell depicts the value of how much the col depends on the row.

Studying these graphs informs us that AGCRN was able to learn some differentiation while GWN was not. Figure 2a shows us the relationships found on the JSE using AGCRN. Looking at the third row and thirteenth column, we see a lime green square indicating that the third node depends more on the thirteenth node than it does from the others. The third Stock corresponds to the Anglo-Gold Ashanti plc and the thirteenth corresponds to Harmony Gold Mining Company Limited. While the model cannot guarantee any real financial relationship, it believes that Harmony Gold's share price depends more on AngloGold's share price than other stocks in the simulation. A similar weight is attached to the thirteenth row, third column, indicating that the pattern found wasn't completely one-way and has some directionality to it, but it should be noted that the weights are asymmetric and may relate to each other differently. The model perceived some relationship between two stocks in the same sector without any classification labels, which serves as rough reason to believe that the model is able to perceive some form of relationships, and the ROI score indicates that the relationships found were effective, even if they might not be real. It should also be noted that whenever the model finds such a relationship it makes the weights of the other nodes smaller, since it

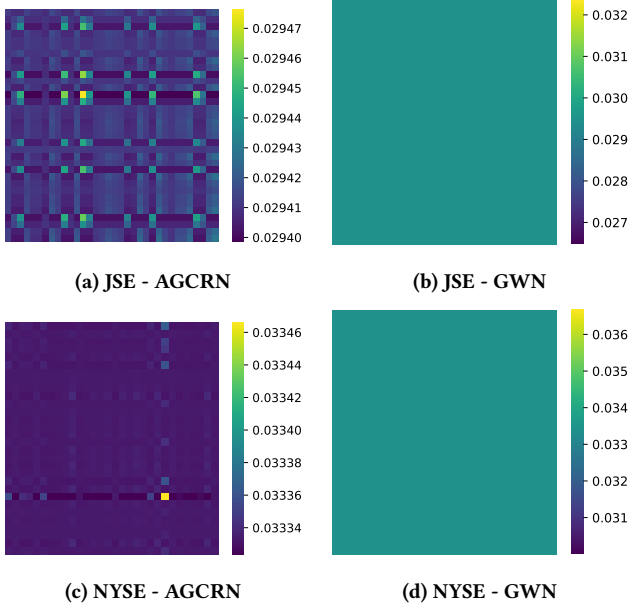


Figure 2: Adjacency Matrix for GWN and AGCRN on the JSE and NYSE

processes weights using Softmax, hence the bands on dark lines that occur as the other stocks after being reduced.

Interestingly we see only two cells with deep yellow. One of them occurs in Figure 2a and the other occurs in Figure 2c. They both occur on the $row_n = col_n$ line, which indicates that the model believes that the other nodes are less important to forecast the return for only that node. One way of quickly affirming this is looking at Figure 2c and focussing on the row and column of the yellow spot. The row values are reduced, hence them being darker, but the column does not have darker bands, indicating that the other nodes still use that node more than it uses them for forecasting. This is not consistent with other values of the identity matrix as not all nodes are more heavily weighted to themselves as these two are, indicating that these two stocks are likely driver nodes that may lead others but that is only speculation.

The GWN on the other hand found no such relationships and opted for uniform weights instead as seen by Figure 2b and Figure 2d. The GWN was supplied with a static graph of $n \times n$ ones to start with so it could slowly adjust them while training, but it opted to focus on temporal patterns and use them while treating all nodes as equally related. This could be due the GWN being overloaded from the static graph which provided initial spatial weights, but it could have decided it was better to focus on the temporal component and did not update the spatial one. Another potential reason this happened is that GWN smooths out gains making all the values extremely close to each other, producing nearly identical weights and its also possible that both of these happened in conjunction.

Finally it is also important to note that although AGCRN did find some relationships, the raw values are very close to each other, indicating that the differences are narrow and the effects of these

relationships is minor. The range of the all four heatmaps in Figure 2 being narrow also indicates that even though fine tuning can indicate some stocks as more beneficial, the encoders still use the entire market for most of the prediction, and the slight weight differences serve as fine tuning.

5.3 Trading Environment Analysis

We also record the number of shares traded for each configuration. Table 4 contains the JSE’s number of shares traded while Table 5 contains the NYSE’s. The number of shares traded across seeds are presented in the format: *mean $\pm$ one standard deviation*.

5.3.1 Market Volatility and Resulting Trading Patterns. The rule baseline had 0 shares traded for each configuration, except the DNN on NYSE where it lost money. This is due to the encoder not outputting values high enough to beat the threshold for the rule-based agent to act upon. The agent is simply not confident in buying or selling and so they end up holding for the entire period. This is one of the main reasons for the need of DRL agents to make decisions that a static threshold cannot. The threshold used for the baseline was 0.35% which was reasoned as the transaction fee, which is 0.25% and some buffer for confidence. We tested the baseline using a threshold of 0.25%, which has no safety buffer and found that the results were the extremely similar except for the DNN on the NYSE, which got worse. (Its data is in the appendix for those interested).

The DRL agents’ number of shares traded observed on the JSE are much smaller than the ones on the NYSE. On the JSE, the model with the smallest number of shares traded was the DNN-PPO model with 19.80 ± 4.40 shares across the entire set, while the model with the highest number of shares was the DNN-A2C model with 71.00 ± 59.99 shares. On the other hand, the lowest number of shares traded on the NYSE was 5062.00 ± 436.83 shares using the DNN-A2C model while the highest count was 14223.20 ± 4307.48 shares with the DNN-DDQN model.

An interesting note is that DNN has the models with the highest and lowest number of shares traded, but none of them have the best ROI, which means they either bought too many shares, without choosing which to purchase properly, and wasted their money or made too few trades and just kept most of their portfolio as cash and didn’t invest properly. Another interesting detail is that the DDQN has many more shares traded than the Actor-Critic models on the NYSE, almost with three times the average number of shares and around nearly ten times the standard deviation for some models. Combined with the fact that DDQN agents were the only ones to not beat the Index, on the NYSE, we can infer that the DDQN model’s strategies that worked during training were likely less effective during the test set. A2C and PPO trade with fewer shares compared to DDQN on the NYSE, and end up with approximately 5,000 shares traded on average. The JSE’s share counts are more uniformly distributed, where DDQN is the largest, for GWN and AGCRN, but it is not up to three times larger like the DDQN on the NYSE was.

This share count might give insight into the type of markets that the JSE and NYSE are without us explicitly providing any context to the agents. The share counts on the JSE are low, which leads us to believe that the models made fewer large transactions with

Encoder	Rule-Based	DDQN	A2C	PPO
DNN	0.00	68.60 $\pm$ 45.52	71.00 $\pm$ 59.99	19.80 $\pm$ 4.40
GWN	0.00	69.40 $\pm$ 19.86	23.00 $\pm$ 10.43	25.60 $\pm$ 4.92
AGCRN	0.00	58.60 $\pm$ 23.41	54.80 $\pm$ 26.48	38.6 $\pm$ 17.68

Table 4: Number of shares traded on the JSE

Encoder	Rule-Based	DDQN	A2C	PPO
DNN	105998.00	14223.20 $\pm$ 4307.48	5062.00 $\pm$ 436.83	5679.20 $\pm$ 856.55
GWN	0.00	13471.80 $\pm$ 4602.64	5219.60 $\pm$ 462.25	5113.60 $\pm$ 467.98
AGCRN	0.00	13763.80 $\pm$ 6526.78	5085.80 $\pm$ 260.31	5200.80 $\pm$ 381.01

Table 5: Number of shares traded on the NYSE

high confidence, and possibly held onto their stocks for some time. We hypothesize this as the number of shares traded is as low as 54.80 ± 26.48 for our best performing model. For comparison, the same model had 5085.80 ± 260.31 shares traded on the NYSE. This stark difference could be the result of the types of market that the JSE and NYSE provide.

Both datasets use the same transaction fee of 0.25% but seem to arrive at vastly different trading strategies as a result of the inherent market dynamics in the JSE and NYSE. We hypothesized that the models on the JSE buy large quantities of shares in fewer number of transactions, while the NYSE models have many more transactions, but have much lower quantity of shares per transaction. If this were true, then JSE models seem to favour buy and hold strategies and simply let their stocks’ value grow, while NYSE models are constantly trading and finding small waves of incremental profits.

This behaviour can also be a product of the 100 shares per stock limit that was implemented. The reason for its implementation was to limit volatility for the model’s own safety and to ensure that it doesn’t make any actions that could influence the market as a whole, but it’s possible that the limit also helped shape the strategies that the models had to make as a result.

5.3.2 Policy Focus and Share Trading Analysis. To validate our hypothesis, we focused on the AGCRN-A2C model. It performed well on both datasets and beat both the JSE Top40 Index and the NYSE Dow Jones Index in terms of ROI. We reran the model evaluation and captured the number of shares bought and sold separately and also captured when shares were bought and sold. This revealed a massive difference in our assumptions and reality proving our hypothesis incorrect. On the NYSE, we found that of the total 5085.80 ± 260.31 shares traded had 5072.60 ± 250.75 shares bought on average and only 13.20 ± 18.43 shares sold on average. On the JSE, we found the AGCRN-A2C model never sold any stocks on any seed, and that the model had only bought and held stocks. With this we were sure that our models were implementing some kind of buy-and-hold strategy. Then we parsed the models actions to view at which timesteps it purchased the shares and when it sold them. We found that the AGCRN-A2C model, across both datasets,

bought all the shares it could at the start of the test set by chipping the balance little by little only buying around one or two hundred shares each timestep. The model would buy the shares using specific weights and maintain that weight as it bought more shares on the following timesteps. The only time the model made any sales, it would use up that money to buy more shares and hold them. The reason that the JSE has so much fewer than the NYSE was because while both agents start with the same balance, the raw prices of the stocks on the JSE were orders of magnitude more expensive, which made the agent instantly use up all its money in the first few timesteps.

In the end, some of our models were not trading everyday as we expected them to, but they had tuned themselves to learn a weighted distribution simply becoming portfolio managers similar to their respective Index.

While this meant the model was great at portfolio management, and perhaps sometimes better than the Index, it also introduced some architectural flaws in the model. The model was used to a static list of stocks. This strategy could work for large number of stocks, but would not easily work for stocks that could change dynamically as that would require the weights to change dynamically which would greatly increase complexity and compute. This also makes it harder to test the robustness of the model in a market crash, where buy and hold would not work. It is likely possible that the model would keep the shares in a crash or sell too slowly and that would plummet the value of the portfolio. It also showed that the 100 shares limit was never reached as the model was just never that confident in any one given stock during the test set, it’s still possible there was an effect during the training or validation set. To investigate even further we plot Figure 3.

Figure 3 shows the models with the best and worst ROI models on each dataset alongside the total amount of shares owned over time. Figure 3a is the best JSE model while Figure 3b shows worst in terms of ROI. Figure 3c is the best NYSE model while Figure 3d shows worst in terms of ROI. For each of the line graphs that track the portfolio value: the bold black line is the mean across the 5 seeds; the light grey band is one standard deviation of the variance across seeds; the dashed red line is 1.0 and represents the initial

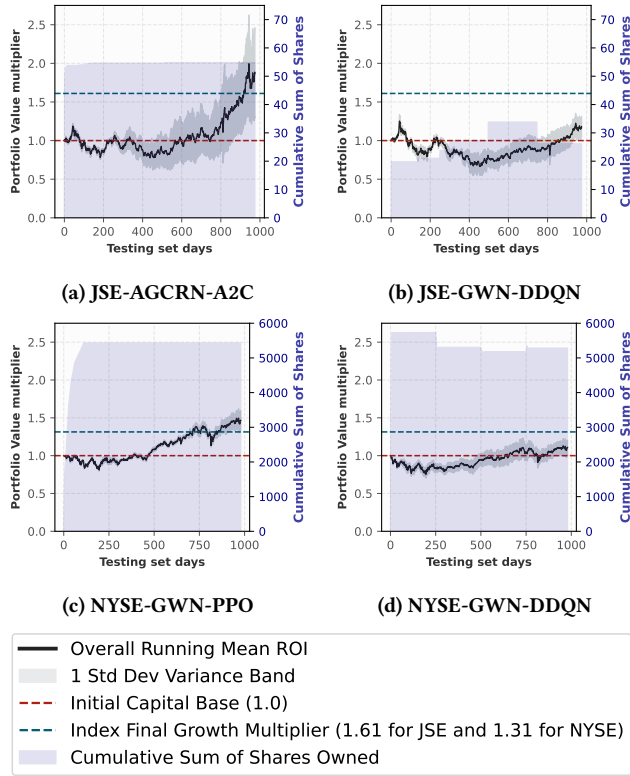


Figure 3: Portfolio value of the models while using walk forward evaluation over the test set and the Shares Owned Count

balance; the dashed teal line is the return multiplier of the Index for the respective dataset at the end of the test set. The light blue filled graph is the number of shares that the model owns.

We can see the AGCRN-A2C model follows a buy and hold rule which we can tell by visually confirming on Figure 3a that the number of shares remains constant after initial buying on the JSE. The same thing can be said for the GWN-PPO model on the NYSE from Figure 3c, where we can see the rise in shares owned more clearly and then it stays uniform till the end, indicating a buy-and-hold strategy for this model as well. We note that GWN-DDQN is the worst ROI model on both datasets and that while it does mostly follow a buy and hold-strategy, it has major shifts that happen at the start/end of each year from Figure 3b and Figure 3d. An interesting detail is that all models seem to experience a downwards spike around day 800 indicating the possibility for a global event. The time frame aligns with when the USA announced tariffs, but that is only speculation at best.

5.4 Limitations

We were only able to manage the four years of walk-forward and ten years of training using the time and compute capabilities we had. Given more resources, we would have considered using a much longer window and retraining models fully with a sliding window over the whole dataset. This would test our model over more kinds

of markets and keep the models relatively stable since we only retrain them from scratch rather than needing to fine tune them to current regimes. We also did not account for dividends, splits or merges which make a significant difference in reality. Given more resources, the environment could be expanded to facilitate these changes or adjusted-values could be used to simulate their effects.

6 Conclusion and Future Work

We found that DRL agents did much better than a simple rule-based threshold trader and even had some models that managed to beat the Index in terms of pure ROI, and one model (AGCRN-PPO) that managed to beat the Index in terms of ROI, Sortino and Calmar on the JSE. We also found a significant difference that emerged as a result of the encoder, and found the DNN to be a valid workable solution, but found the STGNNs to perform better in general on the JSE, while the DNN was sufficient for the NYSE. We had more models which were successful on the NYSE, and only one model which beat the Index across all 3 metrics on the JSE. We believe this is due to the maturity of the NYSE, which makes market trends simpler, hence the reason DNNs can keep up or outperform more complex structures like STGNNs.

This demonstrated that the structure lost during flattening could have been used for higher quality state data. We found GWN and AGCRN also differed in their preferred forecasting method and had widely different results on each dataset. We also found a significant difference between discrete and continuous agents, finding that discrete agents' actions lacked the depth needed to make fine-tuned decisions which gave continuous agents an advantage over them. The discrete agents used ended up as the worst DRL agents and did not manage to beat our Index target for any configuration.

Some of our models settled on portfolio weight management rather than active daily trading, and it ended up the best model on the JSE in terms of pure ROI. This was the AGCRN-A2C model and we found GWN-PPO to be the best on the NYSE dataset, while the worst model was GWN-DDQN across all sets. In general, A2C had higher ROI, while PPO had better Calmar and Sortino scores while still boasting competitive ROI. Finally, we showed the efficacy and the benefits of DRL agents, STGNNs and the two-part model design.

In terms of future work, we can identify a few avenues warranting further research:

- **Dynamic Spatial Topologies:** The models currently depend on stable market nodes, so they need to be tested on dynamically updating nodes and an STGNN or alternative structure that can handle situations such as delistings, merges or even swapping.
- **Hyperparameter Exploration:** Conducting optimization search over the agent parameter spaces could fundamentally enhance baseline policy margins. This includes training parameters such as rollout epochs or trajectory length.

References

- [1] Lei Bai, Lina Yao, Can Li, Xianzhi Wang, and Can Wang. 2020. Adaptive Graph Convolutional Recurrent Network for Traffic Forecasting. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 17804–17815. doi:10.48550/arXiv.2007.02842

- [2] Thomas Fischer and Christopher Krauss. 2018. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research* 270, 2 (2018), 654–669. doi:10.1016/j.ejor.2017.11.054
- [3] Jeffrey Heaton. 2017. Ian Goodfellow, Yoshua Bengio, and Aaron Courville: Deep learning. *Genetic Programming and Evolvable Machines* 19 (10 2017). doi:10.1007/s10710-017-9314-z
- [4] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. 2017. A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem. (06 2017). doi:10.48550/arXiv.1706.10059
- [5] Ieabeling Kaastra and Milton Boyd. 1996. Designing a neural network for forecasting financial and economic time series. *Neurocomputing* 10, 3 (1996), 215–236. Financial Applications, Part II. doi:10.1016/0925-2312(95)00039-9
- [6] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. 2021. FinRL: deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF21)*. ACM, 1–9. doi:10.1145/3490354.3494366
- [7] M. Magdon-Ismail, A. Nicholson, and Y.S. Abu-Mostafa. 1998. Financial markets: very noisy information processing. *Proc. IEEE* 86, 11 (1998), 2184–2195. doi:10.1109/5.726786
- [8] Burton G. Malkiel. 2003. The Efficient Market Hypothesis and Its Critics. *Journal of Economic Perspectives* 17, 1 (March 2003), 59–82. doi:10.1257/089533003321164958
- [9] Volodymyr Mnih, Adria Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. arXiv:1602.01783 [cs.LG] <https://arxiv.org/abs/1602.01783>
- [10] Srivatsa P and Thomas Plötz. 2024. Using Graphs to Perform Effective Sensor-Based Human Activity Recognition in Smart Homes. *Sensors* 24, 12 (2024). doi:10.3390/s24123944
- [11] Kialan Pillay and Deshendra Moodley. 2022. Exploring Graph Neural Networks for Stock Market Prediction on the JSE. In *Artificial Intelligence Research (Communications in Computer and Information Science, Vol. 1494)*. Springer, Cham, 95–110. doi:10.1007/978-3-030-95070-5_7
- [12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] <https://arxiv.org/abs/1707.06347>
- [13] Zezhi Shao, Zhao Zhang, Wei Wei, Fei Wang, Yongjun Xu, Xin Cao, and Christian S. Jensen. 2022. Decoupled Dynamic Spatial-Temporal Graph Neural Network for Traffic Forecasting. arXiv:2206.09112 [cs.LG] <https://arxiv.org/abs/2206.09112>
- [14] Qiguo Sun, Xueying Wei, and Xibei Yang. 2024. GraphSAGE with deep reinforcement learning for financial portfolio optimization. *Expert Systems with Applications* 238 (2024), 122027. doi:10.1016/j.eswa.2023.122027
- [15] Hado van Hasselt, Arthur Guez, and David Silver. 2015. Deep Reinforcement Learning with Double Q-learning. arXiv:1509.06461 [cs.LG] <https://arxiv.org/abs/1509.06461>
- [16] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*. International Joint Conferences on Artificial Intelligence Organization, 1907–1913. doi:10.24963/ijcai.2019/264
- [17] Qingqing Xu. 2025. Capturing Structural Evolution in Financial Markets with Graph Neural Time Series Models. In *Proceedings of the 2025 International Conference on Big Data, Artificial Intelligence and Digital Economy (BDAIE '25)*. Association for Computing Machinery, New York, NY, USA, 219–226. doi:10.1145/3767052.3767086

AI usage statement

Use of Artificial Intelligence in the form of generative large language models like Gemini was limited to proofreading and enhancing scientific prose. No text is directly copied nor is any data presented generated from AI. AI was used to explain theoretical concepts and market dynamics when understanding was insufficient and AI was used to generate code which we were unable to do, such as extracting the weights from the STGNNs to create the Adjacency matrices in Figure 2.

A Appendix

A.1 Action worked examples

For discrete trading agent, given $\mathbf{A}_t = (1, 1, 0)$, and we had 100 of the given currency to spend, we would allocate 50 to the first and

second stocks and buy up to 100 shares or until the agent runs out of money. For continuous trading agent, given $\mathbf{A}_t = (0.3, 0.2, 0)$ and $\mathbf{P}_{Open,t} = (1, 2, 0.5)$, and we had 100 of the given currency to spend, we would temporarily create vector $temp = \mathbf{A}_t \odot \mathbf{P}_{Open,t} = (0.3, 0.4, 0)$ allocate 42.86 to the first and 57.14 for the second stock, which means that the agent buys 42 shares of the first stock and 28 shares of the second stock, leaving the agent’s balance at 2.

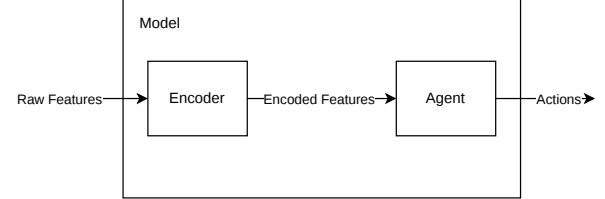


Figure 4: Model Architecture Definition

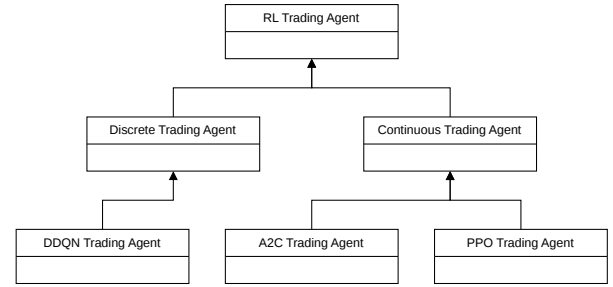


Figure 5: RL agents Architecture Setup

Encoder	JSE		NYSE	
	ROI	Number of shares	ROI	Number of shares
DNN	0.00%	0.00	-19.10%	166381.00
GWN	0.00%	0.00	0.00%	0.00
AGCRN	0.00%	0.00	0.00%	0.00

Table 6: Number of Shares and ROI on the JSE and NYSE for the Rule-based agent, using trading threshold 0.25%